



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS

PROYECTO DE BASE DE DATOS II

SafeBridge: Orquestador Multi-Motor de Respaldos y Validación de Integridad

Estándar de Programación

ANEXO · DEV

CURSO	Base de Datos II
DOCENTE	Ing. Patrick José Cuadros Quiroga
EMPRESA / EQUIPO	BitCraft Solutions
INTEGRANTE	Sierra Ruiz, Iker Alberto (2023077090)
INTEGRANTE	Cortez Mamani, Julio Samuel (2023077283)

*Sistema *SafeBridge**

Estándar de Programación

VERSIÓN 2.0

Control de versiones

Ver-sión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	IASR / JSCM	Ing. P. Cua-dros	Ing. P. Cua-dros	11/06/2026	Versión Original
2.0	IASR / JSCM	Ing. P. Cua-dros	Ing. P. Cua-dros	04/07/2026	Revisión con ejemplos reales del código fuente

Índice General

- 1. Introducción
- 2. Estándares para Backend (Rust)
 - 2.1. Nomenclatura (Naming Conventions)
 - 2.2. Manejo de Errores
 - 2.3. Formateo y Estructura
 - 2.4. Seguridad
- 3. Estándares para Frontend (TypeScript / React)
 - 3.1. Nomenclatura y Tipado
 - 3.2. Estructura de Componentes
 - 3.3. Estilos (Tailwind CSS v4)
- 4. Control de Versiones (Git y Commits)
- 5. Estructura de Archivos

1. Introducción

El presente documento define los estándares de programación, convenciones de nomenclatura y mejores prácticas para el desarrollo del proyecto **SafeBridge**. Al tratarse de una aplicación híbrida usando **Tauri**, se exigen estándares separados pero consistentes para el backend en Rust y el frontend en TypeScript (React).

2. Estándares para Backend (Rust)

El backend de SafeBridge sigue estrictamente las directrices idiomáticas de la comunidad de Rust. Se requiere el uso del linter oficial `clippy` y el formateador `rustfmt`.

2.1. Nomenclatura (Naming Conventions)

- **Variables y Funciones:** `snake_case`. (ej. `let connection_id = ...`, `fn verify_backup()`, `fn emit_log_and_record()`).
- **Structs y Enums:** `UpperCamelCase`. (ej. `struct ConnectionInfo`, `struct BackupResult`, `struct BackupLogPayload`).
- **Constantes y Estáticas:** `SCREAMING_SNAKE_CASE`. (ej. `const FIXED_KEY: &[u8; 32] = ...`).
- **Módulos:** `snake_case`. (ej. `mod connections`, `mod backup`, `mod crypto`).
- **Macros:** Siempre seguidas de `!`. (ej. `println!()`, `format!()`).

2.2. Manejo de Errores

- **No hacer `unwrap()` en producción:** El uso de `.unwrap()` o `.expect()` está prohibido para lógicas críticas que manejen inputs del usuario.
- **Uso de `Result`:** Las funciones que puedan fallar deben retornar un `Result<T, E>`. Ejemplo: `fn encrypt_password(password: &str) -> Result<String, String>`.
- Se permite el operador `?` para propagar errores: `.map_err(|e| format!("Database error: {}", e))?`.
- Los comandos Tauri usan `Result<T, String>` como tipo de retorno estándar.

2.3. Formateo y Estructura

- Mantener los bloques `impl` separados de las definiciones de `struct`.
- Indentación estándar de Rust: 4 espacios (no usar Tabs).
- Ancho máximo de línea: 100 caracteres.
- Un módulo por archivo (`connections.rs`, `backup.rs`, `crypto.rs`, etc.).

2.4. Seguridad

- Las contraseñas **nunca** viajan al frontend: `password: None` en `list_connections`.
- Las contraseñas se cifran antes del INSERT y se descifran solo al instante de ejecución del sidecar.
- Variables de entorno temporales (`PGPASSWORD`, `MYSQL_PWD`) se usan en lugar de argumentos CLI.

3. Estándares para Frontend (TypeScript / React)

3.1. Nomenclatura y Tipado

- **Variables y Funciones TS:** `camelCase`. (ej. `const handleGenerateBackup = () => ...`, `const loadConnections = async () => ...`).

- **Componentes React:** `PascalCase` . (ej. `function ConnectionForm() { ... } , export function Dashboard() { ... }`).
- **Archivos:** `PascalCase.tsx` para componentes React (ej. `Dashboard.tsx` , `ConnectionForm.tsx`). `camelCase.ts` para funciones de utilidad.
- **Tipos e Interfaces:** `PascalCase` usando siempre `interface` o `type` . (ej. `interface LogEntry { ... }`).
- **Estrictez:** No se permite el uso del tipo `any` excepto en interacciones con la API Tauri donde el tipado externo no está definido. TypeScript debe estar en modo estricto (`"strict": true` en el `ts-config.json`).

3.2. Estructura de Componentes

- Emplear Componentes Funcionales y Hooks (`useState` , `useEffect` , `useRef`). Las clases de React están prohibidas.
- Evitar lógica pesada directamente dentro de los componentes visuales; separar en Hooks personalizados cuando sea necesario.
- Estado local con `useState` para componentes simples; para estado compartido, pasarlo como props.

3.3. Estilos (Tailwind CSS v4)

- No usar archivos `.css` puros a menos que sea para variables globales.
- Utilizar clases de utilidad de Tailwind directamente en los componentes.
- Colores del design system: `bg-bg` , `bg-surface` , `text-text` , `text-muted` , `text-accent` , `text-success` , `text-error` , `border-border` .

4. Control de Versiones (Git y Commits)

El proyecto utiliza **Conventional Commits** para generar logs y releases automáticos.

Formato exigido: `<tipo>(<alcance opcional>): <descripción corta>`

Tipos válidos: - `feat:` Una nueva característica. - `fix:` Corrección de un bug. - `docs:` Cambios que solo afectan la documentación. - `style:` Cambios que no afectan el significado del código. - `refactor:` Un cambio de código que no arregla un bug ni añade característica. - `test:` Adición o corrección de pruebas. - `chore:` Actualización de tareas de compilación, gestor de paquetes o configuraciones.

Ejemplos correctos: - `feat(rust): agregar soporte para mysql a tauri shell` - `fix(ui): resolver desbordamiento del botón en historial` - `docs(fd03): añadir requerimientos funcionales completos`

5. Estructura de Archivos

Directorio	Propósito	Convención
<code>src/pages/</code>	Componentes de página completa	<code>PascalCase.tsx</code>
<code>src/components/</code>	Componentes reutilizables	<code>PascalCase.tsx</code>
<code>src-tauri/src/</code>	Módulos Rust del backend	<code>snake_case.rs</code>
Nueva carpeta/	Documentación de informes (FD01-FD05)	<code>FDxx-Nombre.md</code>

Directorio	Propósito	Convención
Nueva carpeta/media/	Imágenes para informes	kebab-case.png

Documento actualizado por el equipo BitCraft Solutions — Universidad Privada de Tacna, FAING-EPIS, Ciclo 2026-I.